

ShopSphere Deployment

Full Submission — Tasks 1–4

Omar Mohammed Elbedawy · EYOUTH-30904291600175

Task 1 — Production Deployment

The app is live and split across three hosts: the React frontend and the Express backend both run on Vercel, and Supabase handles the Postgres database plus file storage for product images. A health-check endpoint on the backend is monitored continuously by UptimeRobot, which pings it on an interval to catch downtime and keep the serverless function warm.

Frontend: <https://kebrete.vercel.app>

Backend: <https://kebrete-backend.vercel.app>

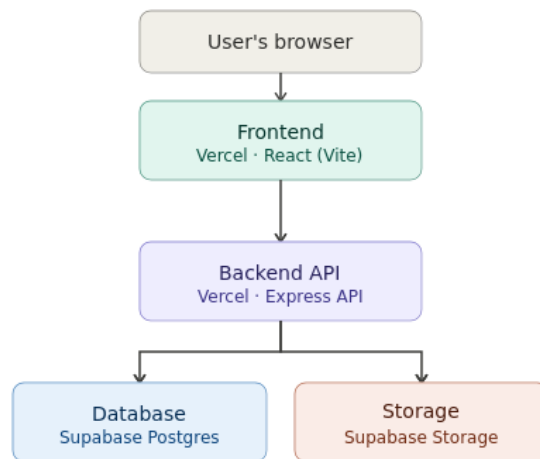
Health-check endpoint: <https://kebrete-backend.vercel.app/health>

Monitoring: UptimeRobot is configured against the health-check endpoint above, pinging it periodically to detect downtime.

Task 2 — Cloud Preparation

2.1 — Architecture diagram

This reflects the actual production path: a request from the browser hits the frontend, which calls the backend API, which in turn reads and writes to Supabase's database and storage.



2.2 — Cloud service classification

I classified each service below by how much of the underlying infrastructure I manage myself versus what the provider takes care of for me.

Service	Model	Reason
Frontend hosting (Vercel)	PaaS	I push my code and Vercel handles the build,

		deployment, scaling, and CDN distribution — I never manage a server.
Backend hosting (Vercel serverless functions)	PaaS	I provide my Express functions and Vercel runs and scales them on demand — there's no OS or infrastructure for me to maintain.
Database & storage (Supabase)	SaaS	I connect through the @supabase/supabase-js client and a connection string to a fully managed Postgres database and storage bucket

2.3 — Multi-cloud namespace simulation

I ran this against a local Kubernetes cluster (Docker Desktop's built-in kind cluster), so it isn't reachable at a public URL. The terminal transcript below is the actual, unedited output from my machine and covers all four required checks: both namespaces created with a frontend pod, backend pod, and a service for each; the frontend service in aws-simulation responding through kubectl port-forward (loaded the nginx welcome page in a browser); and namespace isolation confirmed — the same pod name (frontend-pod) exists independently inside both namespaces, and querying it against the default namespace returns NotFound, proving it does not leak outside its own namespace.

```
PS C:\Users\OMAR\OneDrive\Desktop> kubectl apply -f namespaces.yaml
namespace/aws-simulation created
namespace/gcp-simulation created
pod/frontend-pod created
pod/backend-pod created
service/frontend-service created
service/backend-service created
pod/frontend-pod created
pod/backend-pod created
service/frontend-service created
service/backend-service created

PS C:\Users\OMAR\OneDrive\Desktop> kubectl get all -n aws-simulation
pod/backend-pod      1/1      Running    0          29s
pod/frontend-pod     1/1      Running    0          30s
service/backend-service ClusterIP 10.96.225.93 80/TCP
service/frontend-service ClusterIP 10.96.86.40 80/TCP

PS C:\Users\OMAR\OneDrive\Desktop> kubectl port-forward -n aws-simulation svc/frontend-service 8081:80
Forwarding from 127.0.0.1:8081 -> 80
(browser at localhost:8081 loaded the nginx welcome page)

PS C:\Users\OMAR\OneDrive\Desktop> kubectl get pod frontend-pod -n default
Error from server (NotFound): pods "frontend-pod" not found
```

The manifest file used to create these resources (EYOUTH-30904291600175-ShopSphere-k8s.yaml) is included in the repo.

Task 3 — Application Modernization

I split product reviews out of the main backend into their own service, connected it back to the storefront over REST, added one scheduled serverless function, and documented both decisions below.

3.1 & 3.2 — Review service and REST connection

review-service is a standalone Express + Prisma app with its own package.json and its own deployment on Vercel, completely separate from the main backend. It exposes GET and POST endpoints for reviews per product, plus its own /health check. The main backend's schema has no Review model and runs no review logic — the ShopSphere frontend calls the review service's URL directly to load and submit reviews, so what's shown on a product page genuinely comes from this separate service, not from data sitting in the main app.

Review service: <https://review-service-indol.vercel.app>

Example, verified live: GET /reviews/product/1 → {"reviews":[...],"count":2,"avgRating":4.5}

3.3 — Serverless function

api/daily-summary.js is a separate function inside the review service's repo, outside the Express app's request path entirely. Vercel Cron triggers it once a day; it counts total reviews and computes the average rating, and is protected by a bearer secret so it can't be called by anyone else. I tested it manually and confirmed it runs correctly:

```
Invoke-WebRequest -Uri ".../api/daily-summary" -Headers @{Authorization="Bearer ****"}
StatusCode       : 200
Content          : {"date":"2026-08-15T14:19:10.253Z","totalReviews":2,"avgRating":"4.50"}
```

3.4 — Architecture Decision Record

1. What moved into a microservice — I extracted product reviews out of the main ShopSphere backend and into a standalone service (review-service), with its own Express app, its own Prisma schema, and its own deployment on Vercel at a separate URL.
2. Why reviews specifically — Reviews are a naturally separate concern: they don't participate in checkout, stock, or auth, and they're written far more often than products or orders are. Isolating them means a bug or traffic spike there can't take down checkout, and the service can be redeployed independently of the main app's release cycle.
3. What moved to serverless — The daily review-stats job (api/daily-summary.js), triggered by Vercel Cron on a fixed schedule, authenticated with a bearer secret, running entirely outside any user-facing request.
4. Why this workload fits serverless — It's infrequent (once a day) with no state between runs, so a dedicated always-on server would be wasted capacity. A scheduled serverless function costs nothing when idle and needs no infrastructure of its own.
5. Trade-off accepted — The review service shares the same Supabase Postgres project as the main backend rather than having its own database. This kept hosting simple for a project this size, at the cost of full data-layer independence between the two services.

Task 4 — Production Operations

This task covers the operating practices around the app once it's live: an automated pipeline, structured logs, and a concrete rollback plan.

4.1 — CI/CD pipeline and secrets

The three required environments map onto what's already in place: Development is my local machine, running against its own .env file; Staging is Vercel's automatic Preview deployments (any branch that isn't main); and Production is Vercel's Production deployment tied to main. I built a GitHub Actions workflow

(.github/workflows/deploy-backend.yml) that installs dependencies, generates the Prisma client, and deploys to Vercel production — triggered only on a push to main. It authenticates using VERCEL_TOKEN, VERCEL_ORG_ID, and VERCEL_PROJECT_ID stored as GitHub Actions secrets and referenced only as `{{ secrets.* }}`, never as literal values, and GitHub automatically masks any secret value that appears in the run logs. The existing CI workflow (ci.yml) runs backend and frontend tests on every push and pull request. The main branch is protected by a ruleset requiring both test jobs to pass before a merge is allowed.

4.2 — Structured logging

I added a small JSON logger (utils/logger.js) and two Express middlewares (utils/requestLogger.js): one logs every completed request with method, path, status code, and duration; the other catches unhandled errors and logs them with a stack trace. Every log line is a single JSON object with a timestamp and a severity level (info / warn / error), for example:

```
{ "timestamp": "2026-08-15T16:15:23.968Z", "level": "info", "message": "request completed", "method": "GET", "path": "/products", "statusCode": 200, "durationMs": 120 }
```

These logs are read in production through Vercel's Logs tab for the backend project — every console.log/console.error from a serverless function is captured there automatically, no extra logging service required.

4.3 — Rollback plan

Detection: UptimeRobot pings /health on an interval; a broken deploy causes the endpoint to stop responding or return a non-200 status, triggering an alert within minutes. The structured logs back this up — a spike in 5xx-level request logs or a new unhandled-error entry right after a deploy is an earlier signal than the next uptime check cycle.

Restoration steps: (1) confirm the failure via Vercel's Logs tab and a direct hit to /health; (2) open the affected Vercel project's Deployments tab, where every previous deployment remains available at its own immutable URL; (3) select the last known-good deployment and choose "Promote to Production" (blue and green)— this repoints the production domain instantly, without waiting on a rebuild; (4) in parallel, git revert the breaking commit and push, so main stays in sync with what's actually live; (5) re-check /health and UptimeRobot's status to confirm recovery before closing the incident.

Database changes: if the failing release included a Prisma migration, promoting an old deployment alone isn't sufficient, since old code may not match a changed database shape — the migration would need to be reverted manually via Supabase's SQL editor as well, since Vercel's rollback only affects application code.

4.4 — Project links

Project links

Repository: <https://github.com/omarelbewawy/EYOUTH-30904291600175-ShopSphere.git>

Frontend (application): <https://kebreet.vercel.app>

Review service: <https://review-service-indol.vercel.app>

Backend: <https://kebreet-backend.vercel.app>